



Pets Universe 

Manual Técnico

Este documento constituye el manual técnico oficial de “**Pets Universe**”. Contiene la especificación de la arquitectura del sistema, el diseño de la base de datos, la documentación de APIs y las guías de despliegue y mantenimiento para desarrolladores y administradores.

Realizado por: Vanessa Carolina González Reyes.

Programa: Análisis y Desarrollo de Software.

Ficha: 3227025

Link del repositorio: https://github.com/Thissvane/Pets_Universe



TABLA DE CONTENIDOS

1. INTRODUCCIÓN	X
1.1 Requisitos Técnicos	X
2. Arquitectura y Diagrama	X
2.1. Estructura del Diagrama (Capa por Capa)	X
3. Estandarización de codificación	X
3.1. Convenciones de Nombrado	X
3.2. Script.....	X
3.3 REQUISITOS DE RENDIMIENTO	X
3.4 RESTRICCIONES DE DISEÑO	X



1. Introducción

Con este proyecto, se propone un aplicativo web que permita a los usuarios agendar citas de forma autónoma, acceder al historial de sus mascotas y recibir notificaciones automáticas, mientras optimiza la gestión operativa de la clínica veterinaria, mejorando así la experiencia tanto de los propietarios como del equipo médico. En la actualidad, muchas clínicas veterinarias dependen de métodos tradicionales como llamadas telefónicas y agendas físicas, lo que genera ineficiencias operativas, errores de registro y limitaciones de horario para los clientes.

1.1 Requisitos Técnicos

hardware mínimo recomendado (servidor)

- Procesador: 2 núcleos (2.0 GHz o superior).
- Memoria RAM: mínimo 4 GB (se recomiendan 8 GB).
- Almacenamiento: 20 GB de espacio disponible en disco SSD.
- Red: Conexión a Internet/Intranet estable con ancho de banda mínimo de 10 Mbps.

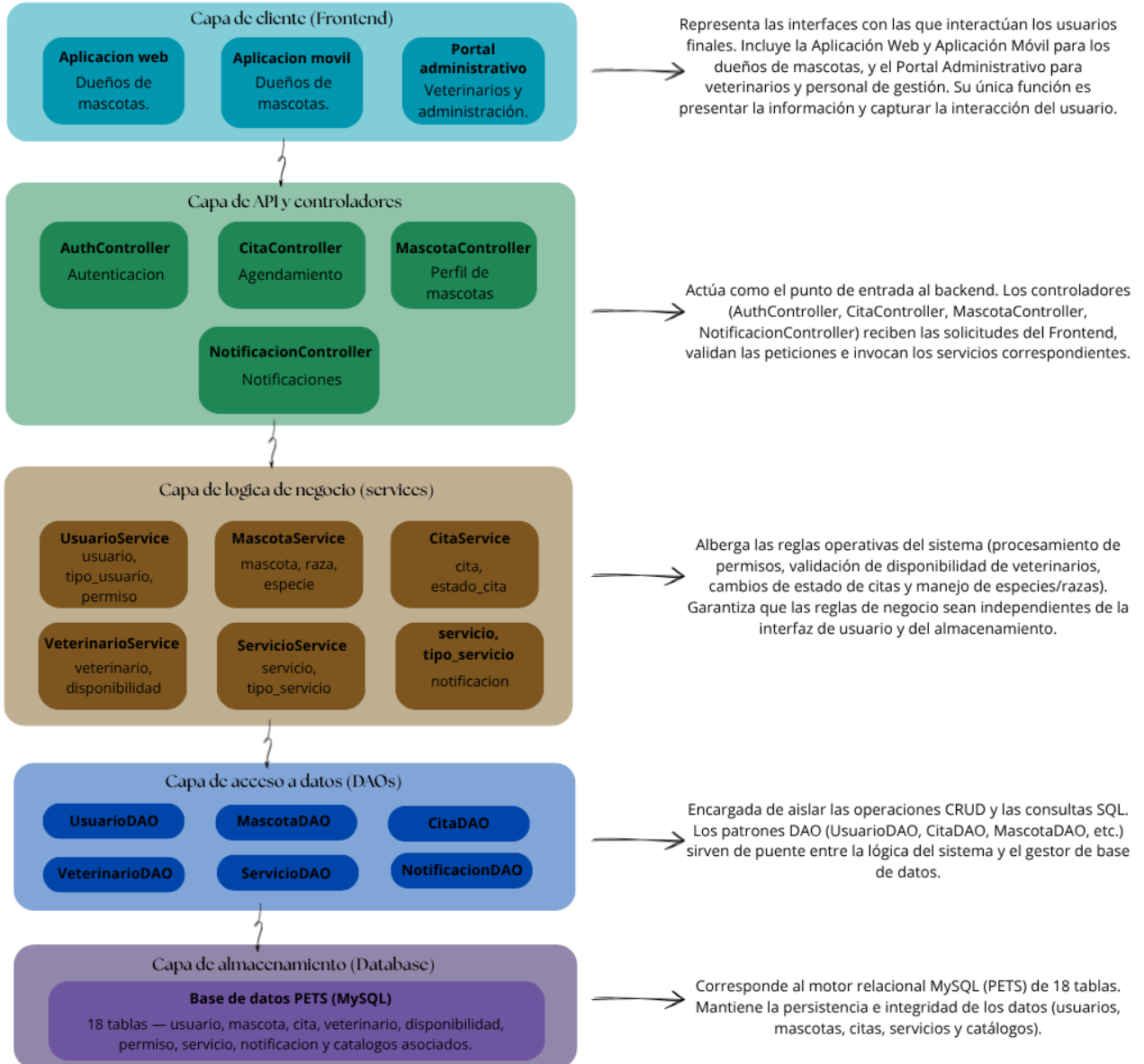
Software y Versiones Compatibles

- Gestor de bases de datos: MySQL 8.0 o superior.
- Entorno de Ejecución Backend: Java Development Kit (JDK 17+) o Python 3.10+.
- Servidor de Aplicaciones / Web: Apache Tomcat 10+ / Nginx / Uvicorn.
- Herramientas de Desarrollo Compatibles: NetBeans IDE / VS Code / MySQL Workbench.
- Navegadores Soportados: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari (últimas 3 versiones).

2. Arquitectura y Diagrama

2.1 Estructura del Diagrama (Capa por Capa)

Se eligió la arquitectura por capas porque **organiza el sistema de forma clara**, separando la interfaz, la lógica y la base de datos para que no se mezclen. Esto permite que la aplicación web, la app móvil y el portal administrativo compartan la misma API sin repetir código. Además, brinda mayor **seguridad** al proteger la base de datos, facilita **cambios futuros** sin dañar otras partes del sistema y permite realizar **pruebas de calidad (QA)** a cada módulo de manera independiente.





3. Estándar de codificación

3.1 Convenciones de Nombrado

- **Clases e Interfaces:** Estilo PascalCase. Ejemplo: TipoNotificacion, EstadoCita, UsuarioDAO.
- **Métodos y Funciones:** Estilo camelCase iniciando con un verbo en infinitivo. Ejemplo: registrarMascota(), obtenerDisponibilidad().
- **Variables:** Estilo camelCase explícito. Ejemplo: idMascota, listaVeterinarios.
- **Objetos de Base de Datos:** Estilo snake_case en minúsculas. Ejemplo: id_mascota, fk_id_usuario.

3.2. Script

```
-- MySQL Workbench Forward Engineering
SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,
FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE,
SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_
O_ZER
O_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

-----
-- Schema PetsUniverse
-----
-----
-- Schema PetsUniverse
-----
-----
DROP SCHEMA IF EXISTS `PetsUniverse` ;
CREATE SCHEMA IF NOT EXISTS `PetsUniverse` DEFAULT CHARACTER SET utf8 ;
USE `PetsUniverse` ;

-----
-- Table `PetsUniverse`.`tipo_documento`
-----
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`tipo_documento` (
  `id_tipo_documento` INT NOT NULL AUTO_INCREMENT,
  `sigla` VARCHAR(45) NOT NULL,
  `descripcion` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`id_tipo_documento`))
ENGINE = InnoDB;
```



-- Table `PetsUniverse`.`tipo\_usuario`

```
-----  
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`tipo_usuario` (  
  `id_tipo_usuario` INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `nombre_tipo` VARCHAR(45) NOT NULL,  
  PRIMARY KEY (`id_tipo_usuario`))  
ENGINE = InnoDB;  
-----
```

-- Table `PetsUniverse`.`usuario`

```
-----  
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`usuario` (  
  `id_usuario` INT NULL AUTO_INCREMENT,  
  `nombre` VARCHAR(45) NOT NULL,  
  `apellido` VARCHAR(45) NOT NULL,  
  `fecha_nacimiento` DATE NOT NULL,  
  `num_documento` VARCHAR(20) NOT NULL,  
  `num_telefono` VARCHAR(10) NOT NULL,  
  `email` VARCHAR(45) NOT NULL,  
  `clave` VARCHAR(255) NOT NULL,  
  `fecha_vencimiento_clave` DATE NOT NULL,  
  `autorizacion_datos` TINYINT NOT NULL,  
  `estado` TINYINT NOT NULL,  
  `tipo_documento_id_tipo_documento` INT NOT NULL,  
  `tipo_usuario_id_tipo_usuario` INT UNSIGNED NOT NULL,  
  PRIMARY KEY (`id_usuario`),  
  UNIQUE INDEX `Clave_UNIQUE` (`clave` ),  
  UNIQUE INDEX `Email_UNIQUE` (`email` ),  
  INDEX `fk_usuario_tipo_documento1_idx`  
  (`tipo_documento_id_tipo_documento` ),  
  INDEX `fk_usuario_tipo_usuario1_idx` (`tipo_usuario_id_tipo_usuario` )  
  ,  
  CONSTRAINT `fk_usuario_tipo_documento1`  
  FOREIGN KEY (`tipo_documento_id_tipo_documento`)  
  REFERENCES `PetsUniverse`.`tipo_documento` (`id_tipo_documento`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION,  
  CONSTRAINT `fk_usuario_tipo_usuario1`  
  FOREIGN KEY (`tipo_usuario_id_tipo_usuario`)  
  REFERENCES `PetsUniverse`.`tipo_usuario` (`id_tipo_usuario`)  
  )  
-----
```



```
ON DELETE NO ACTION
ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-----
-- Table `PetsUniverse`.`especie`
-----
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`especie` (
  `id_especie` INT NOT NULL AUTO_INCREMENT,
  `nombre_especie` VARCHAR(60) NOT NULL,
  `Descripcion` VARCHAR(45) NULL,
  PRIMARY KEY (`id_especie`))
ENGINE = InnoDB;
```

```
-----
-- Table `PetsUniverse`.`raza`
-----
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`raza` (
  `id_raza` INT NULL AUTO_INCREMENT,
  `nombre_raza` VARCHAR(45) NOT NULL,
  `descripcion` VARCHAR(45) NULL,
  `especie_id_especie` INT NOT NULL,
  PRIMARY KEY (`id_raza`),
  INDEX `fk_raza_especie1_idx` (`especie_id_especie` ),
  CONSTRAINT `fk_raza_especie1`
  FOREIGN KEY (`especie_id_especie`)
  REFERENCES `PetsUniverse`.`especie` (`id_especie`)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-----
-- Table `PetsUniverse`.`Mascota`
-----
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`Mascota` (
  `id_Mascota` INT NOT NULL AUTO_INCREMENT,
  `nombre` VARCHAR(20) NOT NULL,
  `peso` DECIMAL(5,2) NOT NULL,
  `usuario_id_usuario` INT NOT NULL,
  `raza_id_raza` INT NOT NULL,
  PRIMARY KEY (`id_Mascota`),
  INDEX `fk_Mascota_usuario1_idx` (`usuario_id_usuario` ),
```



```
INDEX `fk_Mascota_raza1_idx` (`raza_id_raza` ) ,
CONSTRAINT `fk_Mascota_usuario1`
FOREIGN KEY (`usuario_id_usuario`)
REFERENCES `PetsUniverse`.`usuario` (`id_usuario`)
ON DELETE NO ACTION
ON UPDATE NO ACTION,
CONSTRAINT `fk_Mascota_raza1`
FOREIGN KEY (`raza_id_raza`)
REFERENCES `PetsUniverse`.`raza` (`id_raza`)
ON DELETE NO ACTION
ON UPDATE NO ACTION)
ENGINE = InnoDB;

-----

-- Table `PetsUniverse`.`veterinario`
-----

CREATE TABLE IF NOT EXISTS `PetsUniverse`.`veterinario` (
  `id_veterinario` INT NOT NULL AUTO_INCREMENT,
  `num_licencia` VARCHAR(45) NOT NULL,
  `firma_digital` VARCHAR(45) NOT NULL,
  `usuario_id_usuario` INT NOT NULL,
  PRIMARY KEY (`id_veterinario`),
  INDEX `fk_veterinario_usuario1_idx` (`usuario_id_usuario` ) ,
  CONSTRAINT `fk_veterinario_usuario1`
  FOREIGN KEY (`usuario_id_usuario`)
  REFERENCES `PetsUniverse`.`usuario` (`id_usuario`)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION)
ENGINE = InnoDB;

-----

-- Table `PetsUniverse`.`disponibilidad`
-----

CREATE TABLE IF NOT EXISTS `PetsUniverse`.`disponibilidad` (
  `id_disponibilidad` INT NOT NULL AUTO_INCREMENT,
  `dia_semana` INT NOT NULL,
  `hora_inicio` TIME NOT NULL,
  `hora_fin` TIME NOT NULL,
  PRIMARY KEY (`id_disponibilidad`))
ENGINE = InnoDB;

-----
```



-- Table `PetsUniverse`.`estado\_cita`

```
-----  
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`estado_cita` (  
  `id_estado_cita` INT NOT NULL AUTO_INCREMENT,  
  `nombre_estado` VARCHAR(40) NOT NULL,  
  PRIMARY KEY (`id_estado_cita`))  
ENGINE = InnoDB;
```

-- Table `PetsUniverse`.`tipo\_servicio`

```
-----  
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`tipo_servicio` (  
  `id_tipo_servicio` INT NOT NULL AUTO_INCREMENT,  
  `codigo` VARCHAR(20) NOT NULL,  
  `descripcion` VARCHAR(45) NOT NULL,  
  PRIMARY KEY (`id_tipo_servicio`),  
  UNIQUE INDEX `Codigo_UNIQUE` (`codigo` ) )
```

ENGINE = InnoDB;

-- Table `PetsUniverse`.`tipo\_notificacion`

```
-----  
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`tipo_notificacion` (  
  `id_tipo_notificacion` INT NOT NULL AUTO_INCREMENT,  
  `codigo` VARCHAR(20) NOT NULL,  
  `nombre_tipo` VARCHAR(45) NOT NULL,  
  `descripcion` VARCHAR(45) NULL,  
  PRIMARY KEY (`id_tipo_notificacion`),  
  UNIQUE INDEX `Codigo_UNIQUE` (`codigo` ) )  
ENGINE = InnoDB;
```

-- Table `PetsUniverse`.`servicio`

```
-----  
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`servicio` (  
  `id_servicio` INT NOT NULL AUTO_INCREMENT,  
  `descripcion` VARCHAR(45) NOT NULL,  
  `precio` DECIMAL(10,2) NOT NULL,  
  `duracion_estimada` INT NOT NULL,  
  `estado` TINYINT NOT NULL,  
  `tipo_servicio_id_tipo_servicio` INT NOT NULL,
```



```
PRIMARY KEY (`id_servicio`),  
INDEX `fk_servicio_tipo_servicio1_idx`  
(`tipo_servicio_id_tipo_servicio` ) ,  
CONSTRAINT `fk_servicio_tipo_servicio1`  
FOREIGN KEY (`tipo_servicio_id_tipo_servicio`)  
REFERENCES `PetsUniverse`.`tipo_servicio` (`id_tipo_servicio`)  
ON DELETE NO ACTION  
ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

```
-----  
-- Table `PetsUniverse`.`cita`  
-----
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`cita` (  
  `id_cita` INT NOT NULL,  
  `fecha` DATE NOT NULL,  
  `hora` TIME NOT NULL,  
  `status_autorizacion` TINYINT NOT NULL,  
  `observaciones_tesoreria` VARCHAR(200) NOT NULL,  
  `Mascota_id_Mascota` INT NOT NULL,  
  `servicio_id_servicio` INT NOT NULL,  
  `estado_cita_id_estado_cita` INT NOT NULL,  
  `veterinario_id_veterinario` INT NOT NULL,  
  PRIMARY KEY (`id_cita`),  
  INDEX `fk_cita_Mascota1_idx` (`Mascota_id_Mascota` ) ,  
  INDEX `fk_cita_servicio1_idx` (`servicio_id_servicio` ) ,  
  INDEX `fk_cita_estado_cita1_idx` (`estado_cita_id_estado_cita` ) ,  
  INDEX `fk_cita_veterinario1_idx` (`veterinario_id_veterinario` ) ,  
  CONSTRAINT `fk_cita_Mascota1`  
  FOREIGN KEY (`Mascota_id_Mascota`)  
  REFERENCES `PetsUniverse`.`Mascota` (`id_Mascota`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION,  
  CONSTRAINT `fk_cita_servicio1`  
  FOREIGN KEY (`servicio_id_servicio`)  
  REFERENCES `PetsUniverse`.`servicio` (`id_servicio`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION,  
  CONSTRAINT `fk_cita_estado_cita1`  
  FOREIGN KEY (`estado_cita_id_estado_cita`)
```



```
REFERENCES `PetsUniverse`.`estado_cita` (`id_estado_cita`)
ON DELETE NO ACTION
ON UPDATE NO ACTION,
CONSTRAINT `fk_cita_veterinario1`
FOREIGN KEY (`veterinario_id_veterinario`)
REFERENCES `PetsUniverse`.`veterinario` (`id_veterinario`)
ON DELETE NO ACTION
ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- Table `PetsUniverse`.`notificacion`
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`notificacion` (
  `id_notificacion` INT NOT NULL AUTO_INCREMENT,
  `asunto` VARCHAR(200) NOT NULL,
  `mensaje` VARCHAR(255) NOT NULL,
  `tipo_notificacion_id_tipo_notificacion` INT NOT NULL,
  `cita_id_cita` INT NOT NULL,
  PRIMARY KEY (`id_notificacion`),
  INDEX `fk_notificacion_tipo_notificacion1_idx`
  (`tipo_notificacion_id_tipo_notificacion` ),
  INDEX `fk_notificacion_cita1_idx` (`cita_id_cita` ),
  CONSTRAINT `fk_notificacion_tipo_notificacion1`
  FOREIGN KEY (`tipo_notificacion_id_tipo_notificacion`)
  REFERENCES `PetsUniverse`.`tipo_notificacion` (`id_tipo_notificacion`)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION,
  CONSTRAINT `fk_notificacion_cita1`
  FOREIGN KEY (`cita_id_cita`)
  REFERENCES `PetsUniverse`.`cita` (`id_cita`)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- Table `PetsUniverse`.`permisos`
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`permisos` (
  `id_permisos` INT UNSIGNED NOT NULL AUTO_INCREMENT,
  `nombre_permisos` VARCHAR(45) NOT NULL,
```



```
`descripcion` VARCHAR(45) NOT NULL,  
PRIMARY KEY (`id_permisos`),  
UNIQUE INDEX `idPermisos_UNIQUE` (`id_permisos` ) )  
ENGINE = InnoDB;
```

```
-- Table `PetsUniverse`.`Permisos_has_Tipo_usuario`  
-----
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`Permisos_has_Tipo_usuario` (  
  `Permisos_id_permisos` INT UNSIGNED NOT NULL,  
  `Tipo_usuario_id_tipo_usuario` INT UNSIGNED NOT NULL,  
  PRIMARY KEY (`Permisos_id_permisos`, `Tipo_usuario_id_tipo_usuario`),  
  INDEX `fk_Permisos_has_Tipo_usuario_Tipo_usuario1_idx`  
  (`Tipo_usuario_id_tipo_usuario` ) ,  
  INDEX `fk_Permisos_has_Tipo_usuario_Permisos1_idx`  
  (`Permisos_id_permisos` ) ,  
  CONSTRAINT `fk_Permisos_has_Tipo_usuario_Permisos1`  
  FOREIGN KEY (`Permisos_id_permisos`)  
  REFERENCES `PetsUniverse`.`permisos` (`id_permisos`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION,  
  CONSTRAINT `fk_Permisos_has_Tipo_usuario_Tipo_usuario1`  
  FOREIGN KEY (`Tipo_usuario_id_tipo_usuario`)  
  REFERENCES `PetsUniverse`.`tipo_usuario` (`id_tipo_usuario`)  
  ON DELETE NO ACTION  
  ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

```
-- Table `PetsUniverse`.`tipo_usuario_has_permisos`  
-----
```

```
CREATE TABLE IF NOT EXISTS `PetsUniverse`.`tipo_usuario_has_permisos` (  
  `tipo_usuario_id_tipo_usuario` INT UNSIGNED NOT NULL,  
  `permisos_id_permisos` INT UNSIGNED NOT NULL,  
  PRIMARY KEY (`tipo_usuario_id_tipo_usuario`, `permisos_id_permisos`),  
  INDEX `fk_tipo_usuario_has_permisos_permiso1_idx`  
  (`permisos_id_permisos` ) ,  
  INDEX `fk_tipo_usuario_has_permisos_tipo_usuario1_idx`  
  (`tipo_usuario_id_tipo_usuario` ) ,  
  CONSTRAINT `fk_tipo_usuario_has_permisos_tipo_usuario1`  
  FOREIGN KEY (`tipo_usuario_id_tipo_usuario`)
```



```
REFERENCES `PetsUniverse`.`tipo_usuario` (`id_tipo_usuario`)
ON DELETE NO ACTION
ON UPDATE NO ACTION,
CONSTRAINT `fk_tipo_usuario_has_permiso_permiso1`
FOREIGN KEY (`permisos_id_permisos`)
REFERENCES `PetsUniverse`.`permisos` (`id_permisos`)
ON DELETE NO ACTION
ON UPDATE NO ACTION)
ENGINE = InnoDB;
```

```
-- -----
-- Table `PetsUniverse`.`veterinario_has_disponibilidad`
-- -----

CREATE TABLE IF NOT EXISTS
`PetsUniverse`.`veterinario_has_disponibilidad` (
`veterinario_id_veterinario` INT NOT NULL,
`disponibilidad_id_disponibilidad` INT NOT NULL,
PRIMARY KEY (`veterinario_id_veterinario`,
`disponibilidad_id_disponibilidad`),
INDEX `fk_veterinario_has_disponibilidad_disponibilidad1_idx`
(`disponibilidad_id_disponibilidad` ),
INDEX `fk_veterinario_has_disponibilidad_veterinario1_idx`
(`veterinario_id_veterinario` ) ,
CONSTRAINT `fk_veterinario_has_disponibilidad_veterinario1`
FOREIGN KEY (`veterinario_id_veterinario`)
REFERENCES `PetsUniverse`.`veterinario` (`id_veterinario`)
ON DELETE NO ACTION
ON UPDATE NO ACTION,
CONSTRAINT `fk_veterinario_has_disponibilidad_disponibilidad1`
FOREIGN KEY (`disponibilidad_id_disponibilidad`)
REFERENCES `PetsUniverse`.`disponibilidad` (`id_disponibilidad`)
ON DELETE NO ACTION
ON UPDATE NO ACTION)
ENGINE = InnoDB;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
USE `PetsUniverse`;
```



-- 1. tipo_documento

```
INSERT INTO `tipo_documento` (`id_tipo_documento`, `sigla`,  
`descripcion`) VALUES
```

```
(1, 'CC', 'Cédula de Ciudadanía'),  
(2, 'CE', 'Cédula de Extranjería'),  
(3, 'PA', 'Pasaporte'),  
(4, 'PPT', 'Permiso por Protección Temporal');
```

-- 2. tipo_usuario

```
INSERT INTO `tipo_usuario` (`id_tipo_usuario`, `nombre_tipo`) VALUES
```

```
(1, 'Administrador'),  
(2, 'Veterinario'),  
(3, 'Cliente');
```

-- 3. usuario

```
INSERT INTO `usuario` (`id_usuario`, `nombre`, `apellido`,  
`fecha_nacimiento`, `num_documento`, `num_telefono`, `email`, `clave`,  
`fecha_vencimiento_clave`, `autorizacion_datos`, `estado`,  
`tipo_documento_id_tipo_documento`, `tipo_usuario_id_tipo_usuario`)  
VALUES
```

```
(1, 'Ana', 'Gómez', '1985-04-12', '123456789', '3101234567',  
'ana.gomez@mail.com', '$2y$10$abcdefghijklmnopqrstuv1', '2027-04-12',  
1, 1, 1, 1),  
(2, 'Carlos', 'Pérez', '1990-08-22', '987654321', '3209876543',  
'carlos.perez@mail.com', '$2y$10$abcdefghijklmnopqrstuv2',  
'2027-08-22', 1, 1, 1, 2),  
(3, 'María', 'Rodríguez', '1995-12-05', '456789123', '3154567890',  
'maria.rodriguez@mail.com', '$2y$10$abcdefghijklmnopqrstuv3',  
'2027-12-05', 1, 1, 1, 3);
```

-- 4. especie

```
INSERT INTO `especie` (`id_especie`, `nombre_especie`, `Descripcion`) VALUES
```

```
(1, 'Canino', 'Perros domésticos'),  
(2, 'Felino', 'Gatos domésticos');
```

-- 5. raza

```
INSERT INTO `raza` (`id_raza`, `nombre_raza`, `descripcion`,  
`especie_id_especie`) VALUES
```

```
(1, 'Labrador Retriever', 'Perro amigable y activo', 1),  
(2, 'Siamés', 'Gato esbelto y vocal', 2);
```

-- 6. Mascota

```
INSERT INTO `Mascota` (`id_Mascota`, `nombre`, `peso`,
```



```
`usuario_id_usuario`, `raza_id_raza`) VALUES
(1, 'Firulais', 25.50, 3, 1),
(2, 'Michi', 4.20, 3, 2);
-- 7. veterinario
INSERT INTO `veterinario` (`id_veterinario`, `num_licencia`,
`firma_digital`, `usuario_id_usuario`) VALUES
(1, 'VET-98765', 'firma_carlos_perez.png', 2);
-- 8. disponibilidad
INSERT INTO `disponibilidad` (`id_disponibilidad`, `dia_semana`,
`hora_inicio`, `hora_fin`) VALUES
(1, 1, '08:00:00', '12:00:00'),
(2, 2, '14:00:00', '18:00:00');
-- 9. estado_cita
INSERT INTO `estado_cita` (`id_estado_cita`, `nombre_estado`) VALUES
(1, 'Programada'),
(2, 'Completada'),
(3, 'Cancelada');
-- 10. tipo_servicio
INSERT INTO `tipo_servicio` (`id_tipo_servicio`, `codigo`,
`descripcion`) VALUES
(1, 'CONS', 'Consultas y Urgencias'),
(2, 'PREV', 'Medicina Preventiva'),
(3, 'CIRU', 'Cirugía y Procedimientos'),
(4, 'HOSP', 'Hospitalización y Monitoreo'),
(5, 'ESTE', 'Peluquería y Estética'),
(6, 'GUAR', 'Guardería y Hospedaje');
-- 11. tipo_notificacion
INSERT INTO `tipo_notificacion` (`id_tipo_notificacion`, `codigo`,
`nombre_tipo`, `descripcion`) VALUES
(1, 'REM', 'Recordatorio', 'Recordatorio de cita médica');
-- 12. servicio
INSERT INTO `servicio` (`id_servicio`, `descripcion`, `precio`,
`duracion_estimada`, `estado`, `tipo_servicio_id_tipo_servicio`) VALUES
(1, 'Consulta General Caninos', 50000.00, 30, 1, 1),
(2, 'Vacunación Antirrábica', 35000.00, 15, 1, 2),
(3, 'Desparasitación Interna', 25000.00, 15, 1, 2),
(4, 'Esterilización Canina Hembra', 180000.00, 90, 1, 3);
-- 13. cita
INSERT INTO `cita` (`id_cita`, `fecha`, `hora`, `status_autorizacion`,
```



```
`observaciones_tesoreria`, `Mascota_id_Mascota`,
`servicio_id_servicio`, `estado_cita_id_estado_cita`,
`veterinario_id_veterinario`) VALUES
(1, '2026-06-15', '09:00:00', 1, 'Pago verificado exitosamente', 1, 1,
1, 1);
-- 14. notificacion
INSERT INTO `notificacion` (`id_notificacion`, `asunto`, `mensaje`,
`tipo_notificacion_id_tipo_notificacion`, `cita_id_cita`) VALUES
(1, 'Recordatorio de Cita', 'Tiene una cita programada para mañana a
las 9 AM', 1, 1);
-- 15. permisos
INSERT INTO `permisos` (`id_permisos`, `nombre_permisos`,
`descripcion`) VALUES
(1, 'CREAR_CITA', 'Permite agendar citas'),
(2, 'VER_HISTORIAL', 'Permite ver historias clínicas');
-- 16.
INSERT INTO `permisos_has_Tipo_usuario` (`permisos_id_permisos`,
`Tipo_usuario_id_tipo_usuario`) VALUES
(1, 3),
(2, 2);
-- 17. tipo_usuario_has_permisos
INSERT INTO `tipo_usuario_has_permisos`
(`tipo_usuario_id_tipo_usuario`, `permisos_id_permisos`) VALUES
(3, 1),
(2, 2);
-- 18. veterinario_has_disponibilidad
INSERT INTO `veterinario_has_disponibilidad`
(`veterinario_id_veterinario`, `disponibilidad_id_disponibilidad`)
VALUES
(1, 1),
(1, 2);
```

4. Tecnologías y Paquetes

4.1. Backend

- ★ Java Development Kit (JDK): [17.0.10 LTS](#) – Entorno de ejecución y lenguaje para controladores, servicios y DAOs.



- ★ Python Runtime: [3.11.8](#) – Entorno backend para microservicios y scripts auxiliares del sistema.
- ★ FastAPI: [0.110.0](#) – Framework para endpoints API REST asíncronos.
- ★ Pydantic: [2.6.4](#) – Validación estricta de esquemas de datos e integridad del payload.
- ★ SQLAlchemy: [2.0.28](#) – Mapeador objeto-relacional (ORM) para gestión de consultas.
- ★ Uvicorn: [0.28.0](#) – Servidor ASGI para despliegue de APIs en Python.

4.2. Frontend

- ★ **HTML5:** [Standard](#) – Estructuración semántica del Portal Administrativo y la App Web.
- ★ **CSS3:** [Standard](#) – Hojas de estilo para maquetación, paleta corporativa y responsive design.
- ★ **JavaScript (ES6+):** [ECMAScript 2023](#) – Lógica de interacción en cliente, manipulación del DOM y consumo asíncrono.
- ★ **Bootstrap Framework:** [5.3.3](#) – Framework CSS/JS para grillas responsivas, modales y formularios.
- ★ **Bootstrap Icons:** [1.11.3](#) – Librería de iconografía vectorial para la UI.
- ★ **Axios:** [1.6.8](#) – Cliente HTTP para consumo de APIs desde el cliente.

4.3. Base de Datos y Herramientas

- ★ **MySQL Server:** [8.0.36](#) – Motor RDBMS para persistencia relacional de las 17 tablas.
- ★ **MySQL Workbench:** [8.0.36](#) – Modelado ER, administración de esquemas DDL y consultas DML.